Làm việc nhóm hiệu quả cho Lab Day 03

Research Agent Tool Eval · Prompt Engineering & Tool Calling · lucas.ai.vn

> **Lưu ý:** đề bài (README của repo `Day03-Prompt-Engineering-Tool-Calling-Labs-student`) không quy định cách chia việc. Đây là đề xuất, dựa trên cấu trúc repo và các mốc giờ trong đề. Nhóm điều chỉnh theo số người và thế mạnh của mình.

1 Nhìn bài lab trước khi chia việc

Bài lab có **một việc phải làm tuần tự**, còn lại chạy song song được:

Tuần tự (một người giữ)

Vòng tối ưu **v0 → v1 → v2 → v3**. Mỗi vòng chỉ đổi một thứ trong `system_prompt.md` hoặc `tools.yaml`, chạy lại bộ đánh giá rồi ghi điểm trước và sau.

Song song (chia nhiều người)

Tool mới + bài kiểm thử · 10 tình huống đánh giá của nhóm · giao diện + link dùng thử · chat thử 3 lượt · báo cáo Phần A / poster.

> **Vì sao:** Nếu chia việc mà không tách hai loại này, cả nhóm sẽ xếp hàng chờ nhau ở vòng tối ưu, hoặc nhiều người cùng sửa lời nhắc khiến vòng tối ưu mất ý nghĩa. Biết trước việc nào song song được thì mọi người có việc ngay từ phút đầu.

2 Chia vai (gợi ý cho nhóm 4–5 người)

Vai	Việc	File làm chủ	Vì sao tách vai này
1. Chủ lời nhắc	Chạy v0, đặt giả thuyết, sửa v1–v3, ghi nhật ký phiên bản. Là người duy nhất sửa hai file bên cạnh.	`system_prompt.md` `tools.yaml` `version_log.csv`	Mỗi vòng chỉ được một giả thuyết. Một người giữ thì mới biết chắc thay đổi nào làm điểm lên hay xuống.
2. Người đọc log	Đọc `runs/*.json`, gán loại lỗi cho từng tình huống sai, đề xuất giả thuyết cho vai 1.	`analysis/*.csv`	Đọc log tốn thời gian. Tách ra để vai 1 tập trung sửa, và người đọc log ít bị thiên kiến "mình vừa sửa nên chắc đúng".
3. Thợ công cụ	Viết tool mới (`tool.py`, `TOOL.md`), đăng ký trong `tools/__init__.py`, viết ≥ 3 bài kiểm thử, giữ `pytest` xanh.	`tools/<tên>/` `tests/`	Code và kiểm thử của tool không phụ thuộc lời nhắc, nên làm song song được từ đầu. Kiểm thử chạy ngoại tuyến, không tốn credit.

4. Người ra đề	Viết 10 tình huống (5 một lượt, 5 nhiều lượt, ≥ 1 tình huống gọi tool mới), chạy bộ đánh giá nhóm.	`data/eval_group.json`	Đề tốt nhất là đề đo đúng lỗi nhóm gặp. Người này lấy lỗi từ log v0 và tool mới để ra đề.
5. Giao diện + báo cáo	Dựng Streamlit / Vercel, mở Cloudflare Tunnel, chat thử 3 lượt, viết báo cáo Phần A hoặc poster.	`app.py` `transcripts/` `REPORT.md`	Giao diện và Phần A phải có trước 16:30. Không phụ thuộc lời nhắc nên bắt đầu được ngay.

Nhóm 3 người: gộp vai 1 + 2 và vai 4 + 5. Nhóm 2 người: một người giữ vai 1 + 2, người còn lại làm vai 3 + 4 + 5.

3 Mười mẹo, kèm lý do

1. Chỉ một người được sửa `system_prompt.md` và `tools.yaml`.

> **Vì sao:** Đề yêu cầu mỗi phiên bản một giả thuyết, có điểm trước và sau. Hai người cùng sửa thì điểm thay đổi mà không ai biết vì đâu, và nhật ký phiên bản mất giá trị làm bằng chứng. `tools.yaml` còn là chỗ khai báo tool mới, nên dễ đụng nhau nhất.

2. Chạy v0 sớm nhất, trước khi ai sửa gì.

> **Vì sao:** v0 là mốc để so sánh. Không có v0 thì không có "điểm trước" cho v1, và báo cáo Phần B thiếu cột quan trọng nhất. Log của v0 cũng là nguyên liệu cho người ra đề và người đọc log.

3. Tách người đọc log khỏi người sửa lời nhắc.

> **Vì sao:** Người vừa sửa thường muốn thấy mình đúng. Người thứ hai đọc `observed_mismatch` và `failures` sẽ khách quan hơn. Hai việc cũng chạy song song được: một người đọc log vòng trước, một người sửa vòng sau.

4. Thống nhất tên và tham số của tool mới trước khi viết code.

> **Vì sao:** Bộ đánh giá chấm theo đúng tên tool. Tên trong `tools.yaml`, `TOOL_FUNCTIONS` và các tình huống đánh giá phải khớp nhau. Chốt tên sớm thì thợ công cụ, người ra đề và chủ lời nhắc làm song song mà không phải sửa lại.

5. Viết bài kiểm thử cùng lúc với tool, chạy `pytest` trước mỗi lần gộp code.

> **Vì sao:** Bộ đánh giá chỉ đo mô hình có gọi đúng tool không, chứ không đo code của tool có chạy đúng không. Kiểm thử chạy ngoại tuyến nên không tốn credit. `test_tool_contract.py` bắt lỗi lệch tên trước khi lỗi đó làm hỏng cả bộ đánh giá.

6. Ra đề từ lỗi thật trong log, không ra đề theo tưởng tượng.

> **Vì sao:** Mười tình huống của nhóm có giá trị nhất khi đo đúng chỗ agent của nhóm hay sai, và kiểm tra tool mới có được gọi đúng lúc. Đề ra theo tưởng tượng thường quá dễ, chạy lần nào cũng đúng, không giúp sửa gì.

7. Dựng giao diện ngay từ đầu buổi, không đợi agent "tốt".

> **Vì sao:** Giao diện chỉ gọi agent, không phụ thuộc nội dung lời nhắc. Link dùng thử phải có trong báo cáo Phần A trước 16:30. Để đến cuối thì dễ trễ giờ trình diễn.

8. Ghi nhật ký phiên bản ngay sau mỗi lần chạy, kể cả lần làm điểm tụt.

> **Vì sao:** Báo cáo Phần B phải dựa trên log thật, không dựa vào trí nhớ. Một giả thuyết sai cũng là kết quả: nó cho biết hướng nào không hiệu quả. Đợi cuối buổi mới ghi thì quên mã băm và file chạy tương ứng.

9. Mỗi người một nhánh git theo vai, một người gộp; mỗi người tự giữ `.env`.

> **Vì sao:** Nhánh riêng tránh ghi đè công nhau. Để cấm nộp `.env` và API key. Key dán vào chat nhóm hay đẩy lên GitHub có thể bị lộ và bị dùng mất credit.

10. Họp đứng 5 phút ở mỗi mốc giờ của đề.

> **Vì sao:** Các mốc 15:00, 15:30, 16:00, 16:30, 17:30 là lúc các vai cần giao hàng cho nhau, ví dụ tool mới cần được khai báo, tình huống mới cần được chạy. Họp ngắn đúng mốc giúp phát hiện ai đang tắc trước khi trễ giờ trình diễn.

4 Lịch theo mốc giờ của đề

Mốc	Đề yêu cầu	Nhóm nên có
Trước 15:00	Cài đặt, preflight provider, `pytest` chạy được	Mỗi người tự cài xong và có `.env` riêng. Chốt tên tool mới.
15:00	Chạy baseline + dựng UI	Vai 1 chạy v0. Vai 5 bắt đầu giao diện. Vai 3 bắt đầu tool mới.
15:30	Cải tiến v1 + ít nhất 1 tool	Có v1 và bản đầu tool mới. Vai 3 báo vai 1 khai báo tool vào `tools.yaml`.
16:00	Viết tình huống nhóm + cải tiến v2	Vai 4 xong 10 tình huống. Có v2.
16:30	Trình diễn nhóm	Link dùng thử chạy được. Báo cáo Phần A hoặc poster xong.
17:30	Cải tiến v3 + báo cáo Phần B	Có v3, nhật ký đủ v0–v3, Phần B dựa trên log thật. Nộp bài.

5 Trước khi nộp

Kiểm tra bài nộp

☐ `version_log.csv` có đủ v0, v1, v2, v3

☐ `runs/*.json` và `transcripts/` có đủ

☐ `eval_group.json` có 10 tình huống, có tình huống gọi tool mới

☐ `pytest` xanh, tool mới có ≥ 3 bài kiểm thử

☐ Báo cáo có Phần A và Phần B

☐ Không có `.env`, không có key trong code hay báo cáo

Ba lỗi làm việc nhóm hay gặp

- **Nhiều người cùng sửa lời nhắc:** điểm đổi mà không biết vì đâu.
- **Để giao diện và báo cáo đến cuối:** không kịp có link trước giờ trình diễn.
- **Đổi tên tool mà không báo cả nhóm:** phải đổi đủ ba nơi, nếu không bộ đánh giá chấm sai toàn bộ.

Nguồn: README.md và TOOL-SETUP.md của repo VinUni-AI20k/Day03-Prompt-Engineering-Tool-Calling-Labs-student; bộ slide "Từ Chatbot Đến Agentic Agent" (AICB-P1 · Ngày 3). Cách chia vai và các mẹo là đề xuất của lucas.ai.vn, không phải yêu cầu của đề.